

DEVELOPER GUIDE

RW Crew

How the codebase is put together

A walkthrough of the RW Crew application for a developer who is new to the project (or new to development in general): what each part of the code does, how the frontend and backend talk to each other, and how to trace a feature from a click in the browser all the way down to the database and back.

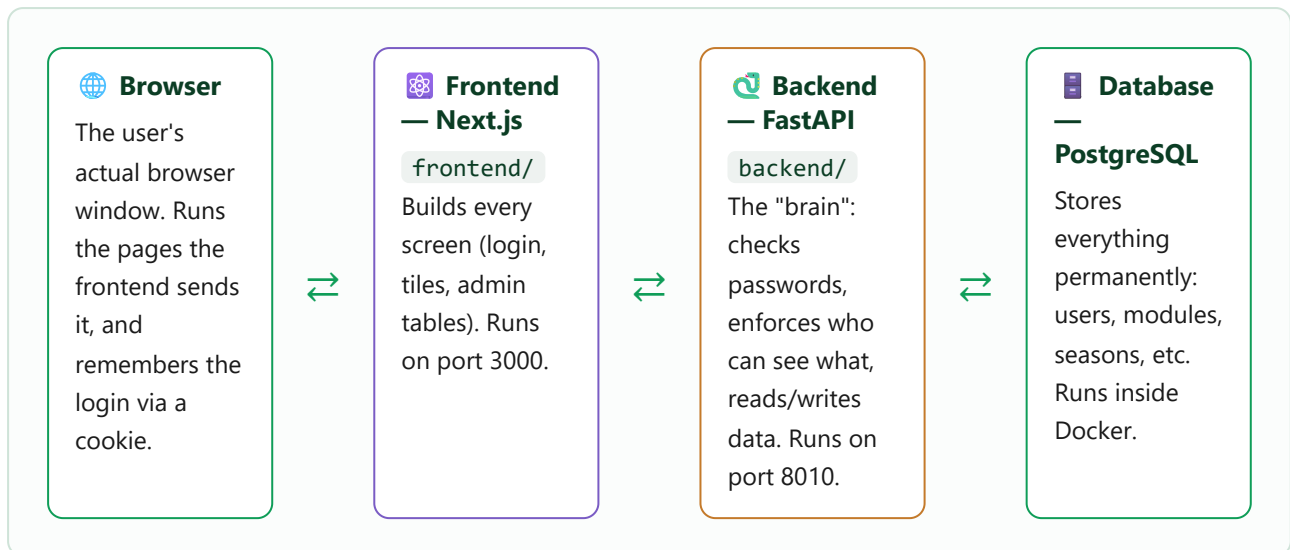
Written for the RW Crew project — `c:\Users\frede\Documents\GitHub\RWCrew`

Table of Contents

- 1** The Big Picture
- 2** Key Concepts, Explained Simply
- 3** Backend Walkthrough (FastAPI)
- 4** Frontend Walkthrough (Next.js)
- 5** Tracing a Request: Logging In
- 6** Tracing a Request: Loading the Landing Page
- 7** Tracing a Request: Changing a User's Access
- 8** The Access Rights Model
- 9** Authentication & Sessions
- 10** The Database & Migrations
- 11** Language & Theme Switching
- 12** Running & Testing Everything
- 13** Where To Go From Here

1. The Big Picture

RW Crew is split into two separate programs that talk to each other over the network, plus a database. This is an extremely common way to build a web application, and once you see it laid out, everything else in this guide will make more sense.



The frontend **never** talks to the database directly. Every single piece of data the browser shows came from asking the backend for it over the network (an "API call"), and the backend is the only thing that ever reads or writes the database. This separation is deliberate: it means all the important rules (e.g. "only a super admin can create a user") live in exactly one place, and can never accidentally be bypassed by the frontend.

Tip: Whenever you're confused about "how does X work", the first question to ask is: *which side is this* — *frontend (what the user sees) or backend (what decides/stores)*? That alone usually tells you which folder to look in.

Why two separate folders?

`frontend/` and `backend/` are two completely independent projects, each with their own dependencies, their own "run" command, and their own programming language (TypeScript for the frontend, Python for the backend). They only relate to each other through plain HTTP requests — the same kind your browser makes when it visits any website. This is why you need **two terminals open** when developing locally: one running the backend, one running the frontend (see section 12).

2. Key Concepts, Explained Simply

These words come up constantly in the code and in this guide. Skim them now, and come back whenever you hit one you forgot.

Endpoint / route

One specific URL the backend understands, tied to one Python function. E.g. `POST /api/auth/login` is an endpoint — a browser (or the frontend) can send a request to that exact address, and one specific function in `app/landing/auth.py` runs to handle it.

Component

A reusable piece of a webpage, written as a function that returns HTML-like code (JSX). E.g.

`ModuleTile` is a component: one function that draws one tile, called 9 times to draw the whole grid.

Server Component vs. Client Component

Next.js has two kinds of components. A **Server Component** (the default) runs on the backend server, before the page is sent to the browser — it can fetch data directly and never sends its own code to the browser. A **Client Component** (marked with `"use client"` at the top of the file) runs inside the browser itself, so it can react to clicks, keep state, etc. Rule of thumb: if a file needs `onClick`, `useState`, or similar, it must be a Client Component.

Props

The "arguments" passed into a component, e.g. `<ModuleTile module={m} hasAccess={true} />` passes two props into `ModuleTile`.

Hook

A special function (always starting with `use`, e.g. `useState`, `useTranslations`) that only works inside Client Components, giving them abilities like "remember a value" or "look up translated text".

ORM & Model

An ORM ("Object-Relational Mapper" — we use SQLAlchemy) lets Python code describe a database table as a normal-looking class instead of writing raw SQL. A "model" (e.g. `class User(Base)` in `app/db/models/user.py`) is one such class — one model = one table.

Schema (Pydantic)

Not the same as a model! A schema describes the exact shape of data going *into* or *out of* an API endpoint (e.g. "a login request must have an email and a password, both text"). FastAPI uses schemas to automatically reject bad requests before your code even runs.

Migration

A small, saved script describing one change to the database's structure (e.g. "add a seasons table"). We use a tool called Alembic to generate and run these, so the database's structure changes in small, trackable, undo-able steps instead of being edited by hand.

JWT (JSON Web Token)

A signed, tamper-proof piece of text that says "this is user #7, and this proof expires at time X". The backend hands one to the browser after a successful login (see section 9).

Cookie

A small piece of data the browser stores and automatically resends with every request to the same website. We store the login tokens in cookies so the browser doesn't have to remember them manually.

Middleware / Proxy

Code that runs *before* a page loads, for every request. RW Crew's `frontend/src/proxy.ts` is one: it decides which language (nl/en) to show, before any page component even runs.

i18n

Short for "internationalization" — the system for showing the same app in multiple languages (Dutch and English here).

3. Backend Walkthrough (FastAPI)

Everything backend-related lives under `backend/app/`. Here's what each folder is for:

Folder	What lives here
<code>core/</code>	Foundational plumbing every other file relies on: settings, the database connection, password hashing, and JWT creation.
<code>db/models/</code>	One file per database table (SQLAlchemy models).
<code>schemas/</code>	One file per "shape of data" used by the API (Pydantic schemas).
<code>landing/</code>	Everything about the landing page's own area: logging in, the super admin's user/access management, and master data.
<code>modules/</code>	One folder per numbered module (module_1 .. module_9), plus the shared logic they all use.
<code>shared/</code>	Small helper functions used by more than one area.
<code>cli/</code>	The <code>seed.py</code> script that fills a brand-new database with starting data.
<code>main.py</code>	The entry point — creates the app and wires every router into it.

`core/config.py` — where settings come from

Defines a `Settings` class listing every configurable value the app needs (database address, secret key, etc.), each with a safe default. On startup, it also reads a `.env` file (a plain text file, never committed to git, listing real values for your machine) and overrides the defaults with whatever it finds there. Every other backend file that needs a setting imports the single shared `settings` object from here — nothing else re-reads the environment itself.

`core/database.py` — the database connection

Creates the `engine` (the object that knows how to actually talk to Postgres) and a `get_db()` function. FastAPI calls `get_db()` automatically for every endpoint that needs the database, handing it a fresh "session" (a temporary workspace for queries) and closing it again afterwards — this pattern is called **dependency injection**, and you'll see `db: Session = Depends(get_db)` as a parameter on almost every endpoint function in the backend.

`core/security.py` — passwords and tokens

Two unrelated jobs live here, because they're both "security-sensitive, small, and shared everywhere":

- `hash_password()` / `verify_password()` — turn a plain password into an unreadable string for storage, and check a login attempt against it. We deliberately use only Python's built-in `hashlib`, not a third-party library (see `CLAUDE.md` for why).

- `create_access_token()` / `create_refresh_token()` / `decode_token()` — create and read the JWTs used to recognise a logged-in user (full explanation in section 9).

db/models/ — one file, one table

Each file defines one Python class that inherits from `Base` (imported from `core/database.py`). SQLAlchemy turns the class's attributes into actual database columns. For example, in

`db/models/user.py`:

```
class User(Base):
    __tablename__ = "landing_users"          # the real table name in Postgres

    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, ...)
    hashed_password: Mapped[str] = mapped_column(String(255), ...)
    is_super_admin: Mapped[bool] = mapped_column(Boolean, default=False, ...)
    ...
```

The five tables today are: `User`, `Module`, `UserModuleAccess`, `ModuleRole`, `RefreshToken` — plus `Season`, the first piece of "master data". `db/base.py` simply imports every single model file in one place, so tools like Alembic (section 10) can discover all of them at once without needing to know the exact list by heart.

schemas/ — the shape of API data

A schema is a class inheriting from Pydantic's `BaseModel`. FastAPI reads these to automatically validate incoming requests and to document what an endpoint returns. Compare a model and its matching schema:

```
class UserCreateRequest(BaseModel):          # schemas/user.py
    email: EmailStr
    password: str = Field(min_length=8)
    display_name: str
    is_super_admin: bool = False
```

If a request to create a user is missing `email`, or its password is only 5 characters, FastAPI rejects it automatically — the endpoint function never even runs. This is why endpoint functions in this codebase look "trusting": by the time your code runs, FastAPI has already checked the data is well-formed.

landing/ — auth, admin, and master data

File	Purpose
<code>deps.py</code>	Defines <code>get_current_user</code> and <code>require_super_admin</code> — reusable checks other endpoints depend on to know "who is calling, and are they allowed to?"
<code>auth.py</code>	Login, logout, refresh, and "who am I" endpoints.
<code>admin.py</code>	Super-admin-only: list/create/delete users, and grant/revoke module access.

<code>master_data.py</code>	Super-admin-only: CRUD for master data like Season.
<code>modules.py</code>	Lists all 9 modules (used to build the landing page's tile grid).

modules/ — one package per numbered module

`modules/common.py` contains one function, `create_module_router(module_key, module_label)`, which builds a complete set of endpoints (a "coming soon" status endpoint, plus role-management endpoints) for *any* module. Each `modules/module_N/router.py` file is then just one line long:

```
# modules/module_1/router.py
from app.modules.common import create_module_router
router = create_module_router(module_key="module-1", module_label="Module 1")
```

This is a good example of avoiding repetition: instead of writing near-identical code 9 times, one shared function is parameterised, and each module just calls it. When Module 1's real functionality is eventually designed, its own endpoints get added directly in `module_1/router.py`, alongside that one line.

main.py — tying it all together

This file does three things, in order: creates the `FastAPI()` app object, adds the CORS middleware (see box below), and calls `app.include_router(...)` once per router file above. Every router already knows its own URL prefix (e.g. `auth_router` only ever handles paths starting with `/api/auth`), so `main.py` doesn't need to know any of the actual endpoint paths — it just needs to know that every router exists.

CORS (Cross-Origin Resource Sharing) is a browser security rule that normally blocks a webpage from calling an API on a different port/domain. Since our frontend (`localhost:3000`) calls our backend (`localhost:8010`) — technically a different "origin" — the backend must explicitly say "requests from localhost:3000 are allowed", which is what `app.add_middleware(CORSMiddleware, ...)` does.

4. Frontend Walkthrough (Next.js)

Next.js uses **file-based routing**: the folder structure under `frontend/src/app/` directly determines the website's URLs. A file named `page.tsx` inside a folder becomes the page shown at that folder's URL.

The route structure

```
src/app/
  layout.tsx           ← the ONE true <html>/<body>, for every page
  [locale]/
    layout.tsx         ← sets up translations for this language
    (auth)/login/page.tsx ← /nl/login or /en/login
    (protected)/
      layout.tsx       ← checks you're logged in, draws sidebar+header
      page.tsx         ← the Mainpage tile grid ( / )
    admin/
      access/page.tsx  ← Manage Access screen
      master-data/page.tsx ← redirects to master-data/season
      master-data/season/page.tsx
    modules/
      module-1/page.tsx ... module-9/page.tsx
```

Two odd-looking things here are intentional Next.js conventions:

- `[locale]` (square brackets) means "this part of the URL is a variable" — it captures whatever language segment was in the URL (`nl` or `en`) so every page underneath can read which language to use.
- `(auth)` and `(protected)` (round brackets) are "route groups" — they organise files into folders *without* becoming part of the actual URL. `(protected)/page.tsx` is simply shown at `/`, not `/protected`.

Why two `layout.tsx` files?

This split trips people up, so it gets its own explanation. `src/app/layout.tsx` is the outer shell: it renders the actual `<html>` tag and sets up dark/light theming. `src/app/[locale]/layout.tsx` sits one level inside it and only sets up translations. They're kept apart specifically so that switching the language (which changes the `[locale]` value) does *not* force the whole page — including the theme — to reload; only the inner, translation-aware part re-renders. See the "Gotchas" section of `CLAUDE.md` for the bug this avoided.

lib/ — shared logic, not tied to any one screen

File	Purpose
<code>config.ts</code>	One constant: the backend's URL.

<code>types.ts</code>	TypeScript shapes matching the backend's Pydantic schemas (kept in sync by hand).
<code>api.ts</code>	The only file allowed to call <code>fetch()</code> from the browser. Every button click that needs the backend calls a function from here.
<code>server-auth.ts</code> , <code>server-api.ts</code>	The same idea as <code>api.ts</code> , but for Server Components — see the box below for why they're separate.
<code>module-theme.ts</code>	Which colour and icon each of the 9 module tiles uses.
<code>utils.ts</code>	Generated by shadcn/ui; just the <code>cn()</code> helper for combining CSS class names.

Why two API-calling files (`api.ts` vs. `server-api.ts`)? A Server Component runs on the Next.js server itself, not in the visitor's browser — so it can't rely on the browser automatically attaching login cookies to its requests. Instead, `server-api.ts` manually reads the incoming request's cookies (via Next.js's `cookies()` function) and forwards them itself. Client Components, which really do run in the browser, use the simpler `api.ts` , which just sets `credentials: "include"` and lets the browser handle cookies automatically.

components/ — organised by who uses them

Folder	Contains
<code>shared/</code>	Used across the whole app: the header (<code>topbar.tsx</code>), the admin sidebar, the theme toggle, the language switcher, the splash screen.
<code>landing/</code>	The login form and the module tile component.
<code>admin/</code>	The Manage Access table and the Season table — the more complex, interactive screens.
<code>ui/</code>	Generated by shadcn/ui (buttons, dialogs, tables, ...). Treat these as a third-party library, not code we hand-wrote.

i18n/ and messages/ — how translations work

`i18n/routing.ts` lists the supported languages (`["nl", "en"]` , Dutch default). `i18n/request.ts` loads the right JSON file from `messages/` for the current request. `i18n/navigation.ts` re-exports language-aware versions of `Link / useRouter` , which is why every component imports `Link` from `@/i18n/navigation` instead of the usual `next/link` — this is what makes navigating around the app automatically stay in the same language.

proxy.ts — the very first thing that runs

Before any page renders, `src/proxy.ts` runs for every request. It uses next-intl's `createMiddleware` to look at the URL and decide the language, redirecting `/` to `/nl` if no language was specified yet.

5. Tracing a Request: Logging In

Reading about files in isolation only gets you so far — the real skill is following one user action through *every* file it touches. Here's exactly what happens when someone types their email/password and clicks "Inloggen":

- 1 **login-form.tsx** (Client Component) — the form's `onSubmit` handler calls `login(email, password)`, a function imported from `lib/api.ts`.
- 2 **lib/api.ts** — `login()` sends a `POST` request with the email/password as JSON to `http://localhost:8010/api/auth/login`.
- 3 **backend/app/landing/auth.py** — the `login()` endpoint function receives it. FastAPI has already checked the JSON matches the `LoginRequest` schema before this function even starts.
- 4 Still in **auth.py** — it looks up the user by email (`db.scalar(select(User)...)`) and calls `verify_password()` from **core/security.py**. If either check fails, it returns a 401 error with a deliberately vague message (so an attacker can't tell whether the email or the password was wrong).
- 5 If correct: **auth.py** calls `create_access_token()` and `create_refresh_token()` (also from **security.py**), saves a hashed copy of the refresh token in the `RefreshToken` table, and attaches both tokens to the response as `HttpOnly` cookies.
- 6 Back in the **browser** — the cookies are now stored automatically (the frontend code never touches them directly). **login-form.tsx** receives a successful response and calls `router.push("/")` to navigate to the Mainpage.
- 7 The Mainpage is a Server Component, so before it can render, the `(protected)/layout.tsx` wrapping it calls `getCurrentUserOnServer()` (**lib/server-auth.ts**), which forwards the browser's cookies to `GET /api/auth/me` to confirm who is logged in — and the tile grid renders.

6. Tracing a Request: Loading the Landing Page

This trace shows how Server Components fetch data *before* anything is sent to the browser — there's no loading spinner, because the page is only sent once it's fully ready.

- 1 Browser requests `/nl`. Next.js runs `(protected)/layout.tsx` first (every page inside it shares this wrapper).
- 2 The layout calls `getCurrentUserOnServer()`. If it returns `null` (not logged in), `redirect("/login")` is called immediately — the actual page component never even runs.
- 3 If logged in, the layout renders the sidebar (only if `is_super_admin`) and the header, then renders `{children}` — which is `(protected)/page.tsx`, the Mainpage itself.
- 4 `page.tsx` calls `serverApiFetch("/api/modules")` (`lib/server-api.ts`) to get the list of all 9 modules, and `getCurrentUserOnServer()` again for the current user's `accessible_module_keys` — Next.js is smart enough to not actually re-fetch this a second time within the same request.
- 5 `page.tsx` loops over the 9 modules and renders one `<ModuleTile />` (`components/landing/module-tile.tsx`) each, passing whether the user has access as a prop — that's what decides whether a tile looks locked or clickable.
- 6 The fully-built HTML is sent to the browser in one go.

7. Tracing a Request: Changing a User's Access

This is a full round trip through every layer, triggered by a super admin ticking a checkbox.

- 1 **access-management.tsx** (Client Component) — the checkbox's `onCheckedChange` calls `handleToggleAccess()`, which works out the user's *complete new* list of module keys (not just the one that changed).
- 2 It calls `setUserModuleAccess(user.id, newModuleKeys)` from **lib/api.ts**, which sends `PUT /api/admin/users/{'{'id{'}}'/access` with `{'{' module_keys: [...] {'}}`.
- 3 **backend/app/landing/admin.py** — the `set_user_module_access()` endpoint runs. Its `Depends(require_super_admin)` parameter means FastAPI already checked the caller is a super admin before this line ever executes — if not, it would already have returned 403 Forbidden.
- 4 The function deletes every existing `UserModuleAccess` row for that user, then adds one new row per module key that was sent — the database ends up matching exactly what the frontend asked for.
- 5 It returns the user's updated summary as JSON. Back in **access-management.tsx**, the local table state is updated immediately (no page reload needed) and a success message ("toast") pops up.

 **Tip:** Notice the pattern repeated in every trace: **Client Component** → **api.ts function** → **HTTP request** → **FastAPI endpoint** → **dependency check** → **database** → **JSON response** → **update the screen**. Once this shape is familiar, reading any new feature in this codebase becomes much faster.

8. The Access Rights Model

There are deliberately **two separate, independent** ways a user's permissions are decided:

	Module access	Module role
Table	<code>UserModuleAccess</code>	<code>ModuleRole</code>
Question it answers	Can this user open this module <i>at all</i> ?	What can they <i>do</i> once inside it (admin/editor/reader)?
Who controls it	Only the super admin, from Manage Access	That module's own admin(s)

A user must have *access* before a *role* means anything — you can't be "the editor of Module 3" if you're not even allowed to open Module 3. The two are stored in completely separate tables on purpose, so each can be checked independently (see `app/modules/common.py`'s `require_module_access` vs. `require_module_admin`).

 **Watch out:** Being the super admin does **not** automatically grant access to every module's content — it only grants the power to hand out access via Manage Access. This is a deliberate design choice (see `app/shared/access.py`) so the super admin's power stays limited to "deciding who can open what".

9. Authentication & Sessions

RW Crew uses two tokens instead of one, which is a standard, more secure pattern than it might first appear:

- **Access token** — short-lived (15 minutes). Sent with every request to prove who you are. Because it expires quickly, if one were ever stolen, the damage window is small.
- **Refresh token** — long-lived (30 days). Only ever used to request a *new* access token, via `POST /api/auth/refresh`, without forcing you to type your password again.

Both are stored in **HttpOnly cookies** — a browser security feature meaning JavaScript running on the page can never read them (only the browser itself can, when automatically attaching them to requests). This protects the tokens even if malicious code somehow ran on the page.

Every time the refresh endpoint is used, the old refresh token is immediately marked `revoked` and a brand new one is issued (called **rotation**). If that same old token is ever presented again, the backend refuses it — a sign it may have been stolen and used by someone else after the real user already refreshed.

10. The Database & Migrations

The database structure (which tables exist, what columns they have) is never edited by hand. Instead, every change is described as a small Python file called a **migration**, stored in

`backend/migrations/versions/`. Alembic (the migration tool) keeps track of which migrations have already been applied to your specific database, in a special table called `alembic_version`.

The usual cycle when you need a schema change

```
# 1. Change (or add) a model file in app/db/models/
# 2. Let Alembic figure out the difference and write a migration for you:
.venv/Scripts/python -m alembic revision --autogenerate -m "describe the change"
# 3. Read the generated file in migrations/versions/ – autogenerate is a
#    helpful first draft, not always perfect, especially for renames.
# 4. Apply it to your actual database:
.venv/Scripts/python -m alembic upgrade head
# 5. Confirm nothing was missed:
.venv/Scripts/python -m alembic check
```

This is exactly how the `Season` table and the `Landing_` table-renaming were added to this project — see the migration files for real examples.

11. Language & Theme Switching

Language: every piece of visible text goes through a translation function (`useTranslations()` in Client Components, `getTranslations()` in Server Components) instead of being typed directly into a component. That function looks the text up by key (e.g. `t("title")`) in `messages/nl.json` or `messages/en.json` depending on the current `[locale]` . Never hard-code user-facing text directly in a component — add it to both JSON files instead.

Theme: handled by the `next-themes` library (`components/shared/theme-provider.tsx`), configured to default to dark mode. It works by adding/removing a `dark` CSS class on the `<html>` tag; `globals.css` defines two full sets of colour variables (one per theme) that automatically apply based on that class.

12. Running & Testing Everything

Two terminals, from the repository root:

```
# Terminal 1 – backend
cd backend
.venv/Scripts/python -m uvicorn app.main:app --reload --port 8010

# Terminal 2 – frontend
cd frontend
npm run dev
```

Then open `http://localhost:3000` — it redirects to `/nl` automatically.

Running the automated tests

```
cd backend
.venv/Scripts/python -m pytest
```

These tests spin up a temporary, empty database in memory (SQLite) for every single test function, so they never touch your real development data, and run in a few seconds. Look at

`backend/tests/landing/test_access.py` for well-commented examples of how a test is structured (create some data directly, log in, call an endpoint, check the result).

 **Tip:** If something that used to work suddenly returns confusing errors, the very first thing to try is restarting both servers completely (stop them, start them again) — this project's `CLAUDE.md` "Gotchas" section documents a couple of real, tricky cases where stale running processes caused exactly this.

13. Where To Go From Here

- Read `CLAUDE.md` at the repository root — it's a shorter, denser reference covering the same ground as this guide, plus a list of specific "gotchas" encountered while building this project (worth reading once you're comfortable with the basics above).
- Pick one small, existing feature (e.g. the Season master data screen) and read every file it touches, start to finish, using the tracing style from sections 5–7 as your template.
- Try making one tiny, safe change yourself — e.g. add a new field to the "New user" dialog — and follow it through both the frontend form and the backend schema, to see the whole path first-hand.
- Whenever a term in the actual code confuses you, check back against the glossary in section 2 first — most confusion in a new codebase comes from unfamiliar vocabulary, not unfamiliar logic.

Generated for the RW Crew project. This document describes the codebase as it exists at the time of writing — as the app grows, prefer reading the actual code and `CLAUDE.md` over this guide if the two ever disagree.